

A3 - Predicting Car Price III

Name: Kan Lin

Student ID: st127119

This assignment is about changing the car price problem into classification.

I use the same car dataset and cleaning steps from my previous assignments. I divide the selling price into four classes and use Multinomial Logistic Regression.

Task 1 - Classification

Import Libraries

I import the libraries that I need for this assignment.

```
In [1]: import os
import json
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
```

Load the Dataset

I load the car dataset and check the first few rows.

```
In [2]: df = pd.read_csv("Cars.csv")

print("Original shape:", df.shape)
df.head()
```

Original shape: (8128, 13)

Out[2]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner_type
--	------	------	---------------	-----------	------	-------------	--------------	------------

0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Individual	Manual	1
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Individual	Manual	5
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	Individual	Manual	1
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	Individual	Manual	1
4	Maruti Swift VXi BSIII	2007	130000	120000	Petrol	Individual	Manual	1

Clean Fuel Column

I remove CNG and LPG cars so the mileage unit is more consistent.

```
In [3]: #Check fuel types
print(df['fuel'].value_counts())

#Remove CNG and LPG cars
df = df[~df['fuel'].isin(['CNG', 'LPG'])].copy()

print("\nFuel types after cleaning:")
print(df['fuel'].value_counts())
```

```
fuel
Diesel    4402
Petrol    3631
CNG         57
LPG         38
Name: count, dtype: int64
```

```
Fuel types after cleaning:
fuel
Diesel    4402
Petrol    3631
Name: count, dtype: int64
```

Clean Owner Column

I change the owner types into numbers and remove Test Drive Cars.

```
In [4]: #Change owner types into numbers
owner_map = {
    'First Owner': 1,
    'Second Owner': 2,
    'Third Owner': 3,
    'Fourth & Above Owner': 4,
    'Test Drive Car': 5
}

df['owner'] = df['owner'].map(owner_map)

#Remove Test Drive Cars
df = df[df['owner'] != 5].copy()

df['owner'].value_counts().sort_index()
```

```
Out[4]: owner
1      5238
2      2073
3       547
4       170
Name: count, dtype: int64
```

Clean Mileage, Engine and Max Power

I remove the text units and change these columns into numbers.

```
In [5]: #Clean mileage
df['mileage'] = pd.to_numeric(
    df['mileage'].str.split().str[0],
    errors='coerce'
)

#Clean engine
df['engine'] = pd.to_numeric(
    df['engine'].str.split().str[0],
    errors='coerce'
)

#Clean max power
df['max_power'] = pd.to_numeric(
    df['max_power'].str.replace('bhp', '', regex=False).str.strip(),
    errors='coerce'
)

df[['mileage', 'engine', 'max_power']].head()
```

```
Out[5]:
```

	mileage	engine	max_power
0	23.40	1248.0	74.00
1	21.14	1498.0	103.52
2	17.70	1497.0	78.00
3	23.00	1396.0	90.00
4	16.10	1298.0	88.20

Create Brand Column

I take the first word from the car name and use it as the brand.

After that I remove the full name and torque columns.

```
In [6]: #Create brand column
df['brand'] = df['name'].str.split().str[0]

#Remove name and torque columns
df = df.drop(columns=['name', 'torque'])

df.head()
```

```
Out[6]:
```

	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mi
0	2014	450000	145500	Diesel	Individual	Manual	1	
1	2014	370000	120000	Diesel	Individual	Manual	2	
2	2006	158000	140000	Petrol	Individual	Manual	3	
3	2010	225000	127000	Diesel	Individual	Manual	1	
4	2007	130000	120000	Petrol	Individual	Manual	1	

Handle Missing Values

I fill the missing numerical values with the median.

```
In [7]: #Fill missing values with median
for column in ['mileage', 'engine', 'max_power', 'seats']:
    df[column] = df[column].fillna(df[column].median())

print("Cleaned shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum())
```

Cleaned shape: (8028, 12)

Missing values:

year	0
selling_price	0
km_driven	0
fuel	0
seller_type	0
transmission	0
owner	0
mileage	0
engine	0
max_power	0
seats	0
brand	0

dtype: int64

Create Four Selling Price Classes

For this assignment I do not predict the exact selling price.

I divide the selling price into four classes: 0, 1, 2 and 3. I use four groups with similar numbers of cars.

```
In [8]: #Create four price groups
df['price_class'], price_bins = pd.qcut(
    df['selling_price'],
    q=4,
    labels=False,
    retbins=True,
    duplicates='drop'
)

df['price_class'] = df['price_class'].astype(int)

print("Price class counts:")
print(df['price_class'].value_counts().sort_index())

print("\nPrice ranges:")
for c in range(4):
    print(
        f"Class {c}: "
        f"{price_bins[c]:,.0f} to {price_bins[c + 1]:,.0f}"
    )
```

```
Price class counts:
price_class
0      2050
1      2044
2      1943
3      1991
Name: count, dtype: int64
```

```
Price ranges:
Class 0: 29,999 to 260,000
Class 1: 260,000 to 450,000
Class 2: 450,000 to 680,000
Class 3: 680,000 to 10,000,000
```

The four classes have similar numbers of cars.

Class 0 is the lowest price group and Class 3 is the highest price group.

```
In [9]: #Plot price class distribution
df['price_class'].value_counts().sort_index().plot(kind='bar')
plt.xlabel('Price Class')
plt.ylabel('Number of Cars')
plt.title('Selling Price Classes')
plt.xticks(rotation=0)
plt.show()
```



Prepare X and y

I separate the input data and the price class.

I remove selling price from X because I already used it to make the price classes.

```
In [10]: #Separate X and y
X = df.drop(columns=['selling_price', 'price_class'])
y = df['price_class'].copy()

#Change text columns into numbers
X = pd.get_dummies(
    X,
    columns=['fuel', 'seller_type', 'transmission', 'brand'],
    dtype=float
)

print("X shape:", X.shape)
print("y shape:", y.shape)

X.head()
```

X shape: (8028, 46)

y shape: (8028,)

```
Out[10]:
```

	year	km_driven	owner	mileage	engine	max_power	seats	fuel_Diesel	1
0	2014	145500	1	23.40	1248.0	74.00	5.0	1.0	
1	2014	120000	2	21.14	1498.0	103.52	5.0	1.0	
2	2006	140000	3	17.70	1497.0	78.00	5.0	0.0	
3	2010	127000	1	23.00	1396.0	90.00	5.0	1.0	
4	2007	120000	1	16.10	1298.0	88.20	5.0	0.0	

5 rows × 46 columns

Split and Scale the Data

I split the data into training data and testing data.

I use 80% for training and 20% for testing. I also scale the input data before training the model.

```
In [11]: #Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

#Scale the data
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("X_train_scaled shape:", X_train_scaled.shape)
print("X_test_scaled shape:", X_test_scaled.shape)
```

X_train_scaled shape: (6422, 46)
X_test_scaled shape: (1606, 46)

Add Intercept and One-Hot Target

I add one intercept column for the model.

I also change the four class labels into one-hot format.

```
In [12]: #Add intercept column
X_train_model = np.c_[
    np.ones(X_train_scaled.shape[0]),
    X_train_scaled
]

X_test_model = np.c_[
    np.ones(X_test_scaled.shape[0]),
    X_test_scaled
]

#Change y into one-hot format
k = 4
y_train_onehot = np.eye(k)[y_train.to_numpy()]

print("Training shape:", X_train_model.shape)
print("Testing shape:", X_test_model.shape)
print("One-hot target shape:", y_train_onehot.shape)
```

Training shape: (6422, 47)
Testing shape: (1606, 47)
One-hot target shape: (6422, 4)

Multinomial Logistic Regression

I use the Logistic Regression class from the class notebook and modify it for this assignment.

I add the required classification metrics and the option to use L2 penalty.

```
In [13]: class LogisticRegression:

    def __init__(
        self,
        k,
        n,
        method="batch",
        alpha=0.05,
```

```

        max_iter=500,
        penalty=None,
        l=0.0,
        random_state=42
    ):
        self.k = k
        self.n = n
        self.method = method
        self.alpha = alpha
        self.max_iter = max_iter
        self.penalty = penalty
        self.l = l
        self.random_state = random_state

    def fit(self, X, Y):
        rng = np.random.default_rng(self.random_state)

        #Start model weights
        self.W = rng.normal(
            0,
            0.01,
            size=(self.n, self.k)
        )

        self.losses = []
        start_time = time.time()

        for i in range(self.max_iter):

            if self.method == "batch":
                X_batch = X
                Y_batch = Y

            elif self.method == "minibatch":
                batch_size = int(0.3 * X.shape[0])

                idx = rng.choice(
                    X.shape[0],
                    size=batch_size,
                    replace=False
                )

                X_batch = X[idx]
                Y_batch = Y[idx]

            elif self.method == "sto":
                idx = rng.integers(0, X.shape[0])

                X_batch = X[idx:idx + 1]
                Y_batch = Y[idx:idx + 1]

            else:
                raise ValueError(
                    'method must be "batch", "minibatch" or "sto"'
                )

```

```

        loss, grad = self.gradient(
            X_batch,
            Y_batch
        )

        self.losses.append(loss)

        #Update model weights
        self.W = self.W - self.alpha * grad

        if i % 100 == 0:
            print(
                f"Loss at iteration {i}: "
                f"{loss:.6f}"
            )

        self.training_time_ = time.time() - start_time

        print(
            "Training time:",
            round(self.training_time_, 3),
            "seconds"
        )

        return self

def gradient(self, X, Y):
    m = X.shape[0]

    #Get predicted probabilities
    h = self.h_theta(X, self.W)

    #Calculate loss
    loss = -np.sum(
        Y * np.log(h + 1e-15)
    ) / m

    #Calculate error
    error = h - Y

    #Calculate gradient
    grad = self.softmax_grad(
        X,
        error
    )

    #Add L2 penalty if selected
    if self.penalty == "l2":
        W_reg = self.W.copy()

        #Do not add penalty to intercept
        W_reg[0, :] = 0.0

        loss += self.l * np.sum(
            W_reg ** 2
        )

```

```

        grad += (
            2.0 *
            self.l *
            W_reg
        )

    return loss, grad

def softmax(self, z):
    #Keep softmax values stable
    z = z - np.max(
        z,
        axis=1,
        keepdims=True
    )

    exp_scores = np.exp(z)

    return exp_scores / np.sum(
        exp_scores,
        axis=1,
        keepdims=True
    )

def softmax_grad(self, X, error):
    return (
        X.T @ error
    ) / X.shape[0]

def h_theta(self, X, W):
    return self.softmax(X @ W)

def predict(self, X):
    return np.argmax(
        self.h_theta(X, self.W),
        axis=1
    )

#Classification metrics

def support(self, y_true):
    y_true = np.asarray(y_true)

    return np.array([
        np.sum(y_true == c)
        for c in range(self.k)
    ])

def accuracy(self, y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    return np.mean(
        y_true == y_pred
    )

```

```

def precision(self, y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    scores = []

    for c in range(self.k):
        tp = np.sum(
            (y_true == c) &
            (y_pred == c)
        )

        fp = np.sum(
            (y_true != c) &
            (y_pred == c)
        )

        denominator = tp + fp

        if denominator == 0:
            score = 0.0
        else:
            score = tp / denominator

        scores.append(score)

    return np.array(scores)

def recall(self, y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    scores = []

    for c in range(self.k):
        tp = np.sum(
            (y_true == c) &
            (y_pred == c)
        )

        fn = np.sum(
            (y_true == c) &
            (y_pred != c)
        )

        denominator = tp + fn

        if denominator == 0:
            score = 0.0
        else:
            score = tp / denominator

        scores.append(score)

    return np.array(scores)

```

```

def f1_score(self, y_true, y_pred):
    precision = self.precision(
        y_true,
        y_pred
    )

    recall = self.recall(
        y_true,
        y_pred
    )

    denominator = precision + recall

    return np.divide(
        2 * precision * recall,
        denominator,
        out=np.zeros_like(precision),
        where=denominator != 0
    )

def macro_precision(self, y_true, y_pred):
    return np.mean(
        self.precision(
            y_true,
            y_pred
        )
    )

def macro_recall(self, y_true, y_pred):
    return np.mean(
        self.recall(
            y_true,
            y_pred
        )
    )

def macro_f1(self, y_true, y_pred):
    return np.mean(
        self.f1_score(
            y_true,
            y_pred
        )
    )

def weighted_precision(self, y_true, y_pred):
    weights = (
        self.support(y_true) /
        len(y_true)
    )

    return np.sum(
        weights *
        self.precision(
            y_true,
            y_pred
        )
    )

```

```

    )
)

def weighted_recall(self, y_true, y_pred):
    weights = (
        self.support(y_true) /
        len(y_true)
    )

    return np.sum(
        weights *
        self.recall(
            y_true,
            y_pred
        )
    )

def weighted_f1(self, y_true, y_pred):
    weights = (
        self.support(y_true) /
        len(y_true)
    )

    return np.sum(
        weights *
        self.f1_score(
            y_true,
            y_pred
        )
    )

def plot(self):
    plt.plot(
        np.arange(len(self.losses)),
        self.losses,
        label="Train Loss"
    )

    plt.title("Training Loss")
    plt.xlabel("Iteration")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()

```

Support

Support means the number of real samples in each class.

For example, if there are 410 Class 0 cars in the test data, the support for Class 0 is 410.

Check My Metrics

I use a small example to compare my precision, recall and F1 functions with scikit-learn.

```
In [14]: #Small example
y_true_mock = np.array([
    0, 0, 0,
    1, 1, 1,
    2, 2, 2,
    3, 3, 3
])

y_pred_mock = np.array([
    0, 1, 0,
    1, 1, 2,
    2, 2, 3,
    3, 0, 3
])

metric_checker = LogisticRegression(
    k=4,
    n=1
)

print("My precision:", metric_checker.precision(y_true_mock, y_pred_mock))
print("My recall:", metric_checker.recall(y_true_mock, y_pred_mock))
print("My F1:", metric_checker.f1_score(y_true_mock, y_pred_mock))

print("\nscikit-learn report:")
print(
    classification_report(
        y_true_mock,
        y_pred_mock,
        labels=[0, 1, 2, 3],
        zero_division=0
    )
)
```

My precision: [0.66666667 0.66666667 0.66666667 0.66666667]

My recall: [0.66666667 0.66666667 0.66666667 0.66666667]

My F1: [0.66666667 0.66666667 0.66666667 0.66666667]

scikit-learn report:

	precision	recall	f1-score	support
0	0.67	0.67	0.67	3
1	0.67	0.67	0.67	3
2	0.67	0.67	0.67	3
3	0.67	0.67	0.67	3
accuracy			0.67	12
macro avg	0.67	0.67	0.67	12
weighted avg	0.67	0.67	0.67	12

```

In [15]: #Check that the results are the same
sk_report_mock = classification_report(
    y_true_mock,
    y_pred_mock,
    labels=[0, 1, 2, 3],
    output_dict=True,
    zero_division=0
)

for c in range(4):
    np.testing.assert_allclose(
        metric_checker.precision(
            y_true_mock,
            y_pred_mock
        )[c],
        sk_report_mock[str(c)]['precision']
    )

    np.testing.assert_allclose(
        metric_checker.recall(
            y_true_mock,
            y_pred_mock
        )[c],
        sk_report_mock[str(c)]['recall']
    )

    np.testing.assert_allclose(
        metric_checker.f1_score(
            y_true_mock,
            y_pred_mock
        )[c],
        sk_report_mock[str(c)]['f1-score']
    )

print("My metric functions match scikit-learn.")

```

My metric functions match scikit-learn.

Train Logistic Regression

First I train Logistic Regression without L2 penalty.

```

In [16]: baseline_model = LogisticRegression(
    k=4,
    n=X_train_model.shape[1],
    method='batch',
    alpha=0.05,
    max_iter=500,
    penalty=None,
    l=0.0,
    random_state=42
)

baseline_model.fit(
    X_train_model,

```

```

    y_train_onehot
)

y_pred_baseline = baseline_model.predict(
    X_test_model
)

print(
    "\nAccuracy:",
    baseline_model.accuracy(
        y_test.to_numpy(),
        y_pred_baseline
    )
)

```

```

Loss at iteration 0: 1.383841
Loss at iteration 100: 0.885335
Loss at iteration 200: 0.806630
Loss at iteration 300: 0.763882
Loss at iteration 400: 0.735441
Training time: 0.439 seconds

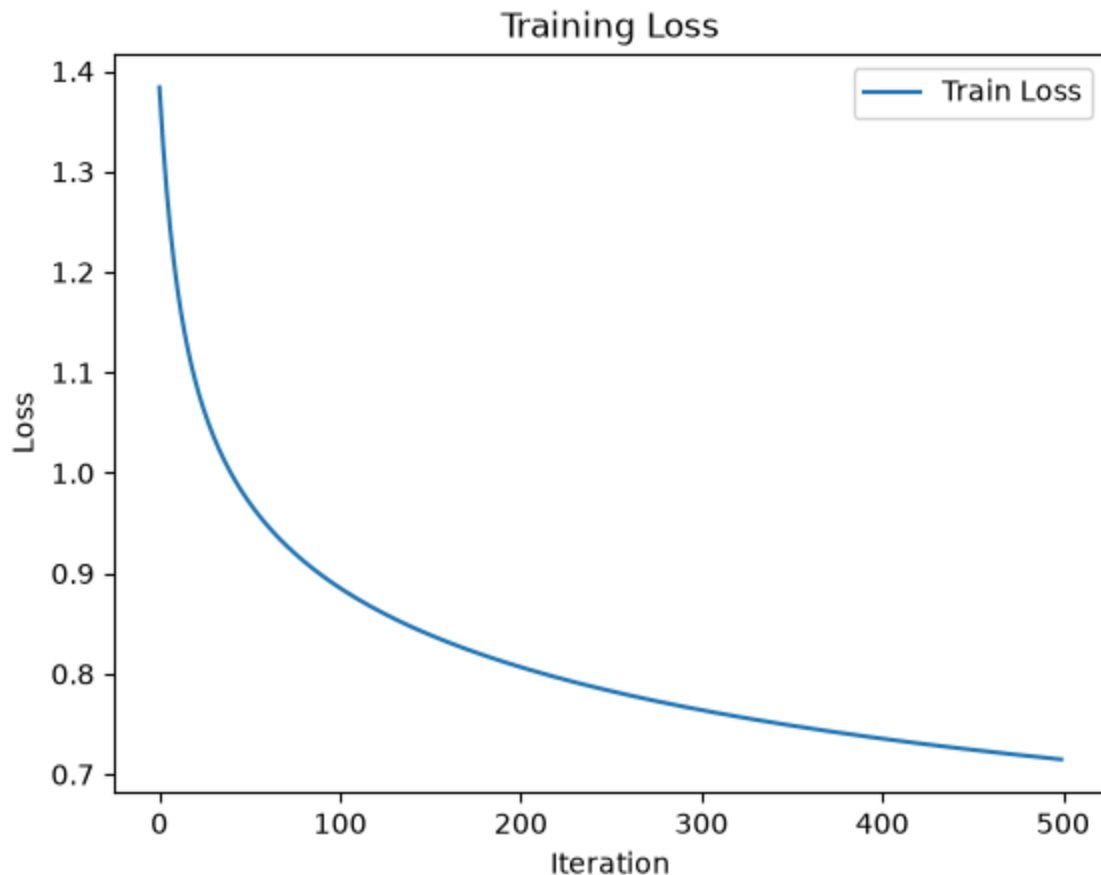
```

```
Accuracy: 0.7222914072229141
```

```

In [17]: #Plot training loss
baseline_model.plot()

```



Check Model Performance

I check accuracy, precision, recall, F1, macro average and weighted average.

```
In [18]: y_test_array = y_test.to_numpy()

precision_values = baseline_model.precision(
    y_test_array,
    y_pred_baseline
)

recall_values = baseline_model.recall(
    y_test_array,
    y_pred_baseline
)

f1_values = baseline_model.f1_score(
    y_test_array,
    y_pred_baseline
)

support_values = baseline_model.support(
    y_test_array
)

custom_report = pd.DataFrame(
    {
        'precision': precision_values,
        'recall': recall_values,
        'f1-score': f1_values,
        'support': support_values
    },
    index=[
        'Class 0',
        'Class 1',
        'Class 2',
        'Class 3'
    ]
)

custom_report
```

```
Out[18]:
```

	precision	recall	f1-score	support
Class 0	0.827103	0.863415	0.844869	410
Class 1	0.664179	0.652812	0.658446	409
Class 2	0.607046	0.575835	0.591029	389
Class 3	0.773956	0.791457	0.782609	398

```
In [19]: print(
    "Accuracy:",
    baseline_model.accuracy(
        y_test_array,
        y_pred_baseline
    )
)
```

```
)

print(
    "Macro Precision:",
    baseline_model.macro_precision(
        y_test_array,
        y_pred_baseline
    )
)

print(
    "Macro Recall:",
    baseline_model.macro_recall(
        y_test_array,
        y_pred_baseline
    )
)

print(
    "Macro F1:",
    baseline_model.macro_f1(
        y_test_array,
        y_pred_baseline
    )
)

print(
    "Weighted Precision:",
    baseline_model.weighted_precision(
        y_test_array,
        y_pred_baseline
    )
)

print(
    "Weighted Recall:",
    baseline_model.weighted_recall(
        y_test_array,
        y_pred_baseline
    )
)

print(
    "Weighted F1:",
    baseline_model.weighted_f1(
        y_test_array,
        y_pred_baseline
    )
)
)
```

Accuracy: 0.7222914072229141
 Macro Precision: 0.7180709381581021
 Macro Recall: 0.7208797830245572
 Macro F1: 0.7192382042494552
 Weighted Precision: 0.7191386816361555
 Weighted Recall: 0.7222914072229141
 Weighted F1: 0.7204790129265781

Compare with scikit-learn

I compare my results with scikit-learn classification report.

```
In [20]: print(
    classification_report(
        y_test_array,
        y_pred_baseline,
        labels=[0, 1, 2, 3],
        digits=4,
        zero_division=0
    )
)
```

	precision	recall	f1-score	support
0	0.8271	0.8634	0.8449	410
1	0.6642	0.6528	0.6584	409
2	0.6070	0.5758	0.5910	389
3	0.7740	0.7915	0.7826	398
accuracy			0.7223	1606
macro avg	0.7181	0.7209	0.7192	1606
weighted avg	0.7191	0.7223	0.7205	1606

```
In [21]: #Confusion matrix
cm = confusion_matrix(
    y_test_array,
    y_pred_baseline,
    labels=[0, 1, 2, 3]
)

plt.figure(figsize=(6, 5))
plt.imshow(cm)

plt.xticks(range(4), [0, 1, 2, 3])
plt.yticks(range(4), [0, 1, 2, 3])

plt.xlabel('Predicted Class')
plt.ylabel('True Class')
plt.title('Baseline Confusion Matrix')

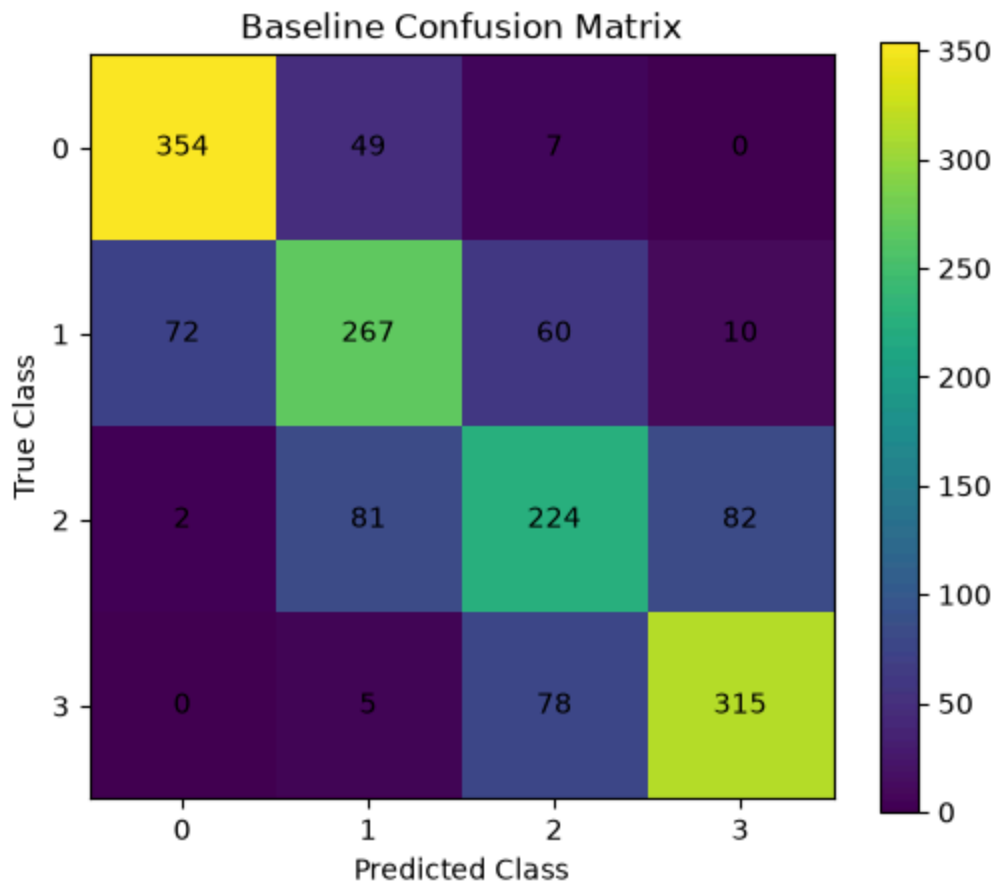
for i in range(4):
    for j in range(4):
        plt.text(
            j,
```

```

        i,
        cm[i, j],
        ha='center',
        va='center'
    )

plt.colorbar()
plt.show()

```



Task 2 - Ridge Logistic Regression

I add L2 penalty to the Logistic Regression model.

I try different lambda values and compare the results.

```

In [22]: experiment_configs = [
    {'Run': 'No penalty', 'Penalty': None, 'Lambda': 0.0},
    {'Run': 'Ridge 0.0001', 'Penalty': 'l2', 'Lambda': 0.0001},
    {'Run': 'Ridge 0.001', 'Penalty': 'l2', 'Lambda': 0.001},
    {'Run': 'Ridge 0.01', 'Penalty': 'l2', 'Lambda': 0.01},
    {'Run': 'Ridge 0.1', 'Penalty': 'l2', 'Lambda': 0.1}
]

trained_models = {}
experiment_rows = []

```

```

for config in experiment_configs:

    print("\n", config['Run'])

    model = LogisticRegression(
        k=4,
        n=X_train_model.shape[1],
        method='batch',
        alpha=0.05,
        max_iter=500,
        penalty=config['Penalty'],
        l=config['Lambda'],
        random_state=42
    )

    model.fit(
        X_train_model,
        y_train_onehot
    )

    prediction = model.predict(
        X_test_model
    )

    experiment_rows.append(
        {
            'Run': config['Run'],
            'Penalty': (
                'None'
                if config['Penalty'] is None
                else config['Penalty']
            ),
            'Lambda': config['Lambda'],
            'Accuracy': model.accuracy(
                y_test_array,
                prediction
            ),
            'Macro Precision': model.macro_precision(
                y_test_array,
                prediction
            ),
            'Macro Recall': model.macro_recall(
                y_test_array,
                prediction
            ),
            'Macro F1': model.macro_f1(
                y_test_array,
                prediction
            ),
            'Weighted F1': model.weighted_f1(
                y_test_array,
                prediction
            ),
            'Final Loss': model.losses[-1],
            'Training Time': model.training_time_
        }
    )

```

```

    )

    trained_models[config['Run']] = model

results = pd.DataFrame(
    experiment_rows
).sort_values(
    by=['Macro F1', 'Accuracy'],
    ascending=False
).reset_index(drop=True)

results

```

No penalty

```

Loss at iteration 0: 1.383841
Loss at iteration 100: 0.885335
Loss at iteration 200: 0.806630
Loss at iteration 300: 0.763882
Loss at iteration 400: 0.735441
Training time: 0.447 seconds

```

Ridge 0.0001

```

Loss at iteration 0: 1.383843
Loss at iteration 100: 0.885608
Loss at iteration 200: 0.807162
Loss at iteration 300: 0.764657
Loss at iteration 400: 0.736446
Training time: 0.456 seconds

```

Ridge 0.001

```

Loss at iteration 0: 1.383855
Loss at iteration 100: 0.888054
Loss at iteration 200: 0.811898
Loss at iteration 300: 0.771512
Loss at iteration 400: 0.745286
Training time: 0.449 seconds

```

Ridge 0.01

```

Loss at iteration 0: 1.383981
Loss at iteration 100: 0.911114
Loss at iteration 200: 0.854189
Loss at iteration 300: 0.829664
Loss at iteration 400: 0.816728
Training time: 0.45 seconds

```

Ridge 0.1

```

Loss at iteration 0: 1.385240
Loss at iteration 100: 1.053477
Loss at iteration 200: 1.048145
Loss at iteration 300: 1.047434
Loss at iteration 400: 1.047299
Training time: 0.45 seconds

```

Out[22]:

	Run	Penalty	Lambda	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F
0	No penalty	None	0.0000	0.722291	0.718071	0.720880	0.719238	0.72047
1	Ridge 0.0001	l2	0.0001	0.722291	0.718071	0.720880	0.719238	0.72047
2	Ridge 0.001	l2	0.0010	0.722291	0.718071	0.720880	0.719238	0.72047
3	Ridge 0.01	l2	0.0100	0.717933	0.713529	0.716553	0.714778	0.71599
4	Ridge 0.1	l2	0.1000	0.675592	0.668152	0.674104	0.669379	0.67055

Select the Best Model

I choose the model with the highest Macro F1 score.

```
In [23]: best_run_name = results.loc[0, 'Run']
best_model = trained_models[best_run_name]

best_prediction = best_model.predict(
    X_test_model
)

print("Best model:", best_run_name)

print(
    "Best Macro F1:",
    best_model.macro_f1(
        y_test_array,
        best_prediction
    )
)

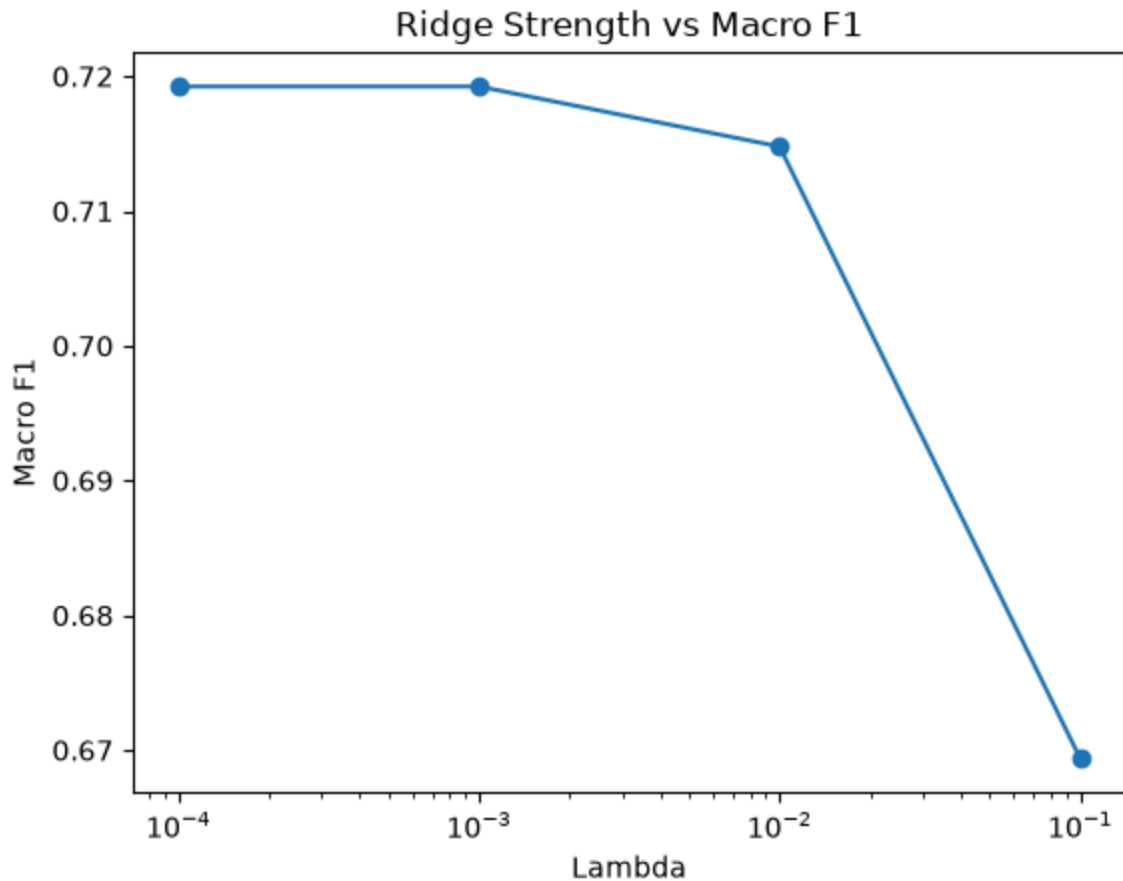
print(
    "Best Accuracy:",
    best_model.accuracy(
        y_test_array,
        best_prediction
    )
)
```

```
Best model: No penalty
Best Macro F1: 0.7192382042494552
Best Accuracy: 0.7222914072229141
```

```
In [24]: #Compare lambda and Macro F1
ridge_results = results[
    results['Penalty'] == 'l2'
].sort_values('Lambda')
```

```
plt.semilogx(
    ridge_results['Lambda'],
    ridge_results['Macro F1'],
    marker='o'
)

plt.xlabel('Lambda')
plt.ylabel('Macro F1')
plt.title('Ridge Strength vs Macro F1')
plt.show()
```



Task 2 Result

I compare Logistic Regression without penalty and with different L2 values.

I use the result table above to choose the best model.

Task 3 - MLflow and Deployment

Now I log the experiment to the class MLflow server.

I use the experiment name `st127119-a3`. I save the best model as `st127119-a3-model`.

I do not log the dataset.

```
In [ ]: #Install MLflow only if it is not already installed
# %pip install "mlflow>=2.9,<3" cloudpickle
```

```
In [ ]: import os
from getpass import getpass

import mlflow
from mlflow.models import infer_signature
from mlflow.tracking import MlflowClient

#Class MLflow login
os.environ['MLFLOW_TRACKING_USERNAME'] = 'admin'

if 'MLFLOW_TRACKING_PASSWORD' not in os.environ:
    os.environ['MLFLOW_TRACKING_PASSWORD'] = getpass(
        'MLflow password: '
    )

MLFLOW_TRACKING_URI = 'http://mlflow.ml.brain.cs.ait.ac.th/'
EXPERIMENT_NAME = 'st127119-a3'
MODEL_NAME = 'st127119-a3-model'

mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
mlflow.set_registry_uri(MLFLOW_TRACKING_URI)
mlflow.set_experiment(EXPERIMENT_NAME)
```

Prepare Model for MLflow

I save the preprocessing and model together so the web app can use normal car information as input.

```
In [ ]: RAW_FEATURE_COLUMNS = [
    'year',
    'km_driven',
    'fuel',
    'seller_type',
    'transmission',
    'owner',
    'mileage',
    'engine',
    'max_power',
    'seats',
```

```

        'brand'
    ]

    NUMERIC_COLUMNS = [
        'year',
        'km_driven',
        'owner',
        'mileage',
        'engine',
        'max_power',
        'seats'
    ]

    CATEGORICAL_COLUMNS = [
        'fuel',
        'seller_type',
        'transmission',
        'brand'
    ]

    numeric_medians = {
        column: float(df[column].median())
        for column in NUMERIC_COLUMNS
    }

    categorical_modes = {
        column: df[column].mode()[0]
        for column in CATEGORICAL_COLUMNS
    }

    feature_columns = X.columns.tolist()

class CarPricePyfuncModel(mlflow.pyfunc.PythonModel):

    def __init__(
        self,
        classifier,
        scaler,
        feature_columns,
        numeric_medians,
        categorical_modes,
        price_bins
    ):
        self.classifier = classifier
        self.scaler = scaler
        self.feature_columns = feature_columns
        self.numeric_medians = numeric_medians
        self.categorical_modes = categorical_modes
        self.price_bins = np.asarray(
            price_bins,
            dtype=float
        )

    def predict(
        self,

```

```

        context,
        model_input
    ):
        frame = pd.DataFrame(
            model_input
        ).copy()

        frame = frame[
            RAW_FEATURE_COLUMNS
        ].copy()

        for column, median in self.numeric_medians.items():
            frame[column] = (
                pd.to_numeric(
                    frame[column],
                    errors='coerce'
                )
                .fillna(median)
            )

        for column, mode in self.categorical_modes.items():
            frame[column] = (
                frame[column]
                .fillna(mode)
                .astype(str)
            )

        encoded = pd.get_dummies(
            frame,
            columns=CATEGORICAL_COLUMNS,
            dtype=float
        )

        encoded = encoded.reindex(
            columns=self.feature_columns,
            fill_value=0.0
        )

        scaled = self.scaler.transform(
            encoded
        )

        model_X = np.c_[
            np.ones(scaled.shape[0]),
            scaled
        ]

        classes = (
            self.classifier
            .predict(model_X)
            .astype(int)
        )

        lower_bounds = np.array([
            self.price_bins[c]
            for c in classes

```

```

    ])

    upper_bounds = np.array([
        self.price_bins[c + 1]
        for c in classes
    ])

    return pd.DataFrame(
        {
            'price_class': classes,
            'lower_bound': lower_bounds,
            'upper_bound': upper_bounds
        }
    )

```

```

In [ ]: #Check the saved model format before MLflow
best_wrapper = CarPricePyfuncModel(
    classifier=best_model,
    scaler=scaler,
    feature_columns=feature_columns,
    numeric_medians=numeric_medians,
    categorical_modes=categorical_modes,
    price_bins=price_bins
)

sample_input = df[
    RAW_FEATURE_COLUMNS
].head(5).copy()

sample_output = best_wrapper.predict(
    None,
    sample_input
)

sample_output

```

Log Experiments to MLflow

I log the model settings and results for each experiment.

```

In [ ]: logged_runs = {}

result_lookup = (
    results
    .set_index('Run')
    .to_dict(orient='index')
)

sample_input = df[
    RAW_FEATURE_COLUMNS
].head(5).copy()

for run_name, model in trained_models.items():

```

```

row = result_lookup[run_name]

wrapper = CarPricePyfuncModel(
    classifier=model,
    scaler=scaler,
    feature_columns=feature_columns,
    numeric_medians=numeric_medians,
    categorical_modes=categorical_modes,
    price_bins=price_bins
)

output_example = wrapper.predict(
    None,
    sample_input
)

signature = infer_signature(
    sample_input,
    output_example
)

with mlflow.start_run(
    run_name=run_name
) as run:

    mlflow.log_params(
        {
            'method': model.method,
            'alpha': model.alpha,
            'max_iter': model.max_iter,
            'penalty': (
                'none'
                if model.penalty is None
                else model.penalty
            ),
            'lambda': float(model.l)
        }
    )

    mlflow.log_metrics(
        {
            'accuracy': float(row['Accuracy']),
            'macro_precision': float(row['Macro Precision']),
            'macro_recall': float(row['Macro Recall']),
            'macro_f1': float(row['Macro F1']),
            'weighted_f1': float(row['Weighted F1']),
            'final_loss': float(row['Final Loss']),
            'training_time': float(row['Training Time'])
        }
    )

    #Save model only
    mlflow.pyfunc.log_model(
        artifact_path='model',
        python_model=wrapper,
        input_example=sample_input,

```

```

        signature=signature
    )

    logged_runs[run_name] = run.info.run_id

    print(
        run_name,
        '->',
        run.info.run_id
    )

```

Register the Best Model

I register the best model and move it to Staging.

```

In [ ]: best_run_id = logged_runs[
        best_run_name
    ]

    best_model_uri = (
        f'runs://{best_run_id}/model'
    )

    registered_model = mlflow.register_model(
        model_uri=best_model_uri,
        name=MODEL_NAME
    )

    client = MlflowClient()

    client.transition_model_version_stage(
        name=MODEL_NAME,
        version=registered_model.version,
        stage='Staging'
    )

    print('Registered model:', MODEL_NAME)
    print('Version:', registered_model.version)
    print('Stage: Staging')

```

Deployment Files and CI/CD

I prepare the files for the web app and GitHub.

I use a README file, an app folder, two unit tests and GitHub Actions. The tests check the model input and output shape.

GitHub Actions runs the tests when I push the code. If the tests pass, deployment can continue.